

Collection Framework

1. ArrayList

1. Create an `ArrayList` of integers and write a method to remove all even numbers from the list.
 2. Write a program to store student names in an `ArrayList` and print them in reverse order without using built-in reverse methods.
 3. Given an `ArrayList` of strings, write a method to find and return the longest string.
 4. Write a program to merge two `ArrayList<Integer>` into a single sorted list without using `Collections.sort()`.
 5. Create an `ArrayList` of custom objects (e.g., `Book` with `title` and `author`) and print all book titles by a given author.
-

2. Map

1. Create a `HashMap<String, Integer>` to count the frequency of characters in a given string.
 2. Write a program to store student IDs and names in a `HashMap` and print all students whose names start with a particular letter.
 3. Using a `TreeMap<Integer, String>`, store key-value pairs and print them in ascending key order.
 4. Write a method that takes two `Map<String, Integer>` and returns a merged map with summed values for matching keys.
 5. Implement a program to find the key with the highest value in a `HashMap<String, Integer>`.
-

3. Set – 5 Questions

1. Use a `HashSet<Integer>` to find all duplicate integers from an array.
 2. Write a program that takes two sets of strings and returns their intersection.
 3. Create a `LinkedHashSet` to store names in insertion order and print them.
 4. Using a `TreeSet<String>`, store a list of words and print them in alphabetical order.
 5. Write a method to remove all elements from a set that satisfy a given condition (e.g., remove all strings longer than 5 characters).
-

4. Comparator and Comparable – 5 Questions

1. Create a `Person` class with `name` and `age`. Implement `Comparable` to sort by age ascending.
2. Write a `Comparator` to sort a list of `Person` objects by name alphabetically.
3. Given a list of `Employee` objects with `salary` and `id`, implement a comparator to sort by salary descending, and if equal, by id ascending.
4. Write a program to sort an array of strings by their length using a `Comparator`.
5. Implement a custom `Comparator` for a `Product` class to sort products by price, then by rating (both ascending).